Inheritance: Building on Your Code's Foundation

WIX1002 Fundamentals of Programming

Every Story Starts with a Blueprint: The Class

The Blueprint

Before inheritance, we need a base. In Object-Oriented Programming (OOP), a **class** is a **blueprint** for **creating objects**. It defines the **data (fields)** and **behaviors** (methods) that its objects will have.

Key Idea

A class is a template; an object is a specific instance created from that template.

W11E01.java

```java
// A simple blueprint for a person.
class SimplePersonE01 {
    private final String name;
    private final int age;

    SimplePersonE01(String name, int age) {
        this.name = name;
        this.age = age;
    }

    void introduce() {
        System.out.println("Hi, I am " + name + " and I am " + age + " years old.");
    }
}

// Creating two instances (objects) from the blueprint.
SimplePersonE01 first = new SimplePersonE01("Aisyah", 19);
SimplePersonE01 second = new SimplePersonE01("Ben", 21);
first.introduce(); // Output: Hi, I am Aisyah and I am 19 years old.
```

The Instances

Extending the Blueprint with `extends`

What is Inheritance?

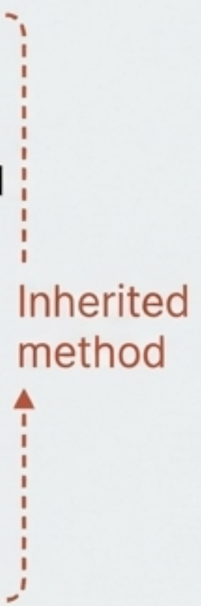
Inheritance is the process of creating a new class (the **subclass** or *derived class*) from an existing class (the **superclass** or *base class*). The subclass automatically inherits the public and protected members of its superclass.

Code Reuse. We don't have to rewrite what the superclass already provides.

- **Superclass / Base Class / Parent**: The class being inherited from (PersonE02).
- **Subclass / Derived Class / Child**: The new class that inherits (StudentE02).

W11E02.java

```
class PersonE02 {
    protected final String name;
    // ... constructor ...
    void sayHello() { // Behavior to be reused
        System.out.println("Hello, I'm " + name + ".");
    }
}
class StudentE02 extends PersonE02 { // The 'extends' keyword
    private final String major;
    // ... constructor ...
    void shareMajor() { // New, specialized behavior
        System.out.println("I study " + major + ".");
    }
}

// Usage
StudentE02 student = new StudentE02("Chong", "CS");
student.sayHello();     // Inherited from PersonE02!
student.shareMajor();   // Defined in StudentE02
```



Building in Layers: Constructor Chaining

Initializing the Superclass

When a subclass object is created, its superclass's constructor must run first to initialize the inherited fields. We use the `super()` keyword to explicitly call a superclass constructor.

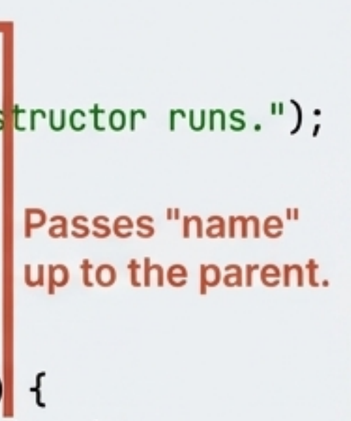
> The call to `super()` must be the very first statement in the subclass's constructor.

Execution Order

1. Superclass constructor runs.
2. Subclass constructor runs.

W11E03.java

```java
class PersonE03 {
  protected final String name;
  PersonE03(String name) {
    this.name = name;
    System.out.println("1. PersonE03 constructor runs.");
  }
}

class StudentE03 extends PersonE03 {
  private final int intakeYear;
  StudentE03(String name, int intakeYear) {
    super(name); // Call PersonE03's constructor
    this.intakeYear = intakeYear;
    System.out.println("2. StudentE03 constructor runs.");
  }
  //...
}

// Output when new StudentE03(...) is created:
// 1. PersonE03 constructor runs.
// 2. StudentE03 constructor runs.
```



Specializing Behavior with Method Overriding

Providing a New Implementation

Sometimes a subclass needs to provide a more specific version of a method it inherits. This is called **method overriding**. The subclass redefines the method with the same signature (name and parameters).

The @Override Annotation: This is a best practice. It tells the compiler you intend to override a method. If you make a mistake (e.g., misspell the method name), the compiler will give you an error.

Dynamic Dispatch: Java automatically calls the most specific version of the method at runtime, even if the object is stored in a superclass variable.

W11E04.java

```
class PersonE04 {
    // ...
    void describe() {
        System.out.println("Person: " + name);
    }
}

class StudentE04 extends PersonE04 {
    // ...
    @Override // Signal intent to override
    void describe() {
        System.out.println("Student: " + name + " (" + major + ")");
    }
}

PersonE04 studentAsPerson = new StudentE04("Farid", "Math");
studentAsPerson.describe(); // Calls the StudentE04 version!
```

Dynamic dispatch: The object's actual type determines which method runs.

Enhancing, Not Replacing: Using `super.method()`

Collaborating with the Superclass

Overriding doesn't mean you have to discard the superclass's logic entirely. You can call the superclass's version of the method from within the overridden method using `super.methodName()`.

This is perfect for adding specialized details while still leveraging the general behavior defined in the parent class.

W11E05.java

```java
class PersonE05 {
    // ...
    void describe() {
        System.out.println("Name: " + name);
    }
}

class StudentE05 extends PersonE05 {
    // ...
    @Override
    void describe() {
        super.describe(); // Call the PersonE05 version first
        // Now add the new, specific information
        System.out.println("Major: " + major);
        System.out.println("CGPA : " + cgpa);
    }
}

// Output of student.describe():
// Name: Gita
// Major: Physics
// CGPA : 3.8
```

The Rules of Overriding

Rule 1: `final` Methods Cannot Be Overridden

If a superclass method is marked with the `final` keyword, it is considered a fixed, unchangeable behavior. Subclasses inherit it but cannot override it.

```
class BaseRuleE08 {
  final protected void safetyRule() { /* ... */ }
}
class ChildRuleE08 extends BaseRuleE08 {
  // Cannot override safetyRule(). Causes a compile error.
}
```

Rule 2: The Return Type Can Be More Specific

The return type of an overridden method can be a subclass of the original return type. This is called a *covariant return type*.

```
class BaseClass { public Employee getValue() { /*...*/ } }
class DerivedClass extends BaseClass {
  @Override
  public HourlyEmployee getValue() { /*...*/ } // OK!
}
```

Rule 3: Access Level Cannot Be More Restrictive

You can make an overridden method *more* accessible (e.g., `protected` to `public`), but not *less* accessible (e.g., `public` to `private`).

`private void method()` in superclass -> `public void method()` in subclass is okay (but it's a new method, not an override). `public void method()` -> `private void method()` is an error.

Controlling Access: Who Can See What?

Access Modifiers

Access modifiers control the visibility of fields and methods.

- **public**: Accessible from any class, anywhere.
- **protected**: Accessible within its own class, by subclasses, and by other classes in the same package. This is key for inheritance.
- **private**: Accessible **only** within its own class. Subclasses cannot access private members of their superclass directly.
- **Default (Package-Private)**: No modifier. Accessible only by classes in the same package.

W11E07.java

```
class VaultE07 {
  public void open() { /*..*/ }
  protected void audit() { /*...*/ }
  private void resetCode() { /*...*/ }
}

class SmartVaultE07 extends VaultE07 {
  public void someMethod() {
    open();
    audit();
    // resetCode(); // COMPILE ERROR!
It's private to VaultE07
  }
}
```

The Payoff: One Interface, Many Forms

Polymorphism

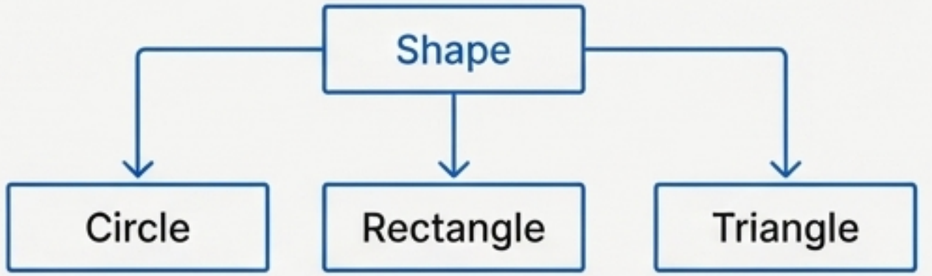
Polymorphism is the ability of an object to take on many forms. In practice, it means you can use a superclass reference to hold a subclass object. This allows us to write flexible code that works with a family of related types without needing to know the specific type at compile time.

The Problem

Imagine you need to manage a collection of different geometric shapes. They all have an “area,” but the calculation is different for each.

The Solution

Create an `abstract` base class `Shape` that declares an `area()` method. Each concrete shape will `extend` `Shape` and provide its own implementation of `area()`.




```
// An abstract "contract" for all shapes
abstract class ShapeE10 {
    abstract double area();
    abstract String name();
}

// Concrete implementations
class CircleE10 extends ShapeE10 {
    @Override double area() { /* πr² */ }
    //...
}

class RectangleE10 extends ShapeE10 {
    @Override double area() { /* width * height */ }
    //...
}
```

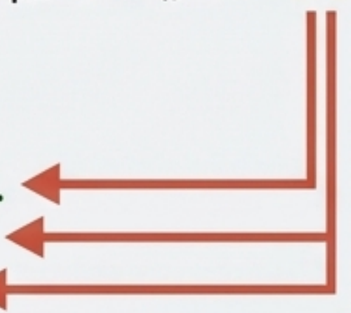
Polymorphism in Action

A Single Loop for All Shapes

Because `Circle`, `Rectangle`, and `Triangle` are all descendants of `Shape`, we can store them in a single array of type `Shape[]`. When we iterate through the array and call `shape.area()`, Java's dynamic dispatch mechanism looks at the *actual* object's type at runtime and calls the correct, overridden `area()` method for that specific shape.

W11E10.java

```
// An array of the superclass type holds various subclass objects.
ShapeE10[] shapes = {
    new CircleE10(2.5),
    new RectangleE10(4, 3),
    new TriangleE10(5, 2)
};

// One loop works for all shapes!
for (ShapeE10 shape : shapes) {
    // Java calls the correct area() at runtime
    System.out.println(shape.name() + " area = " + shape.area());
}

// --- Output ---
// Circle area = 19.63...
// Rectangle area = 12.0
// Triangle area = 5.0
```

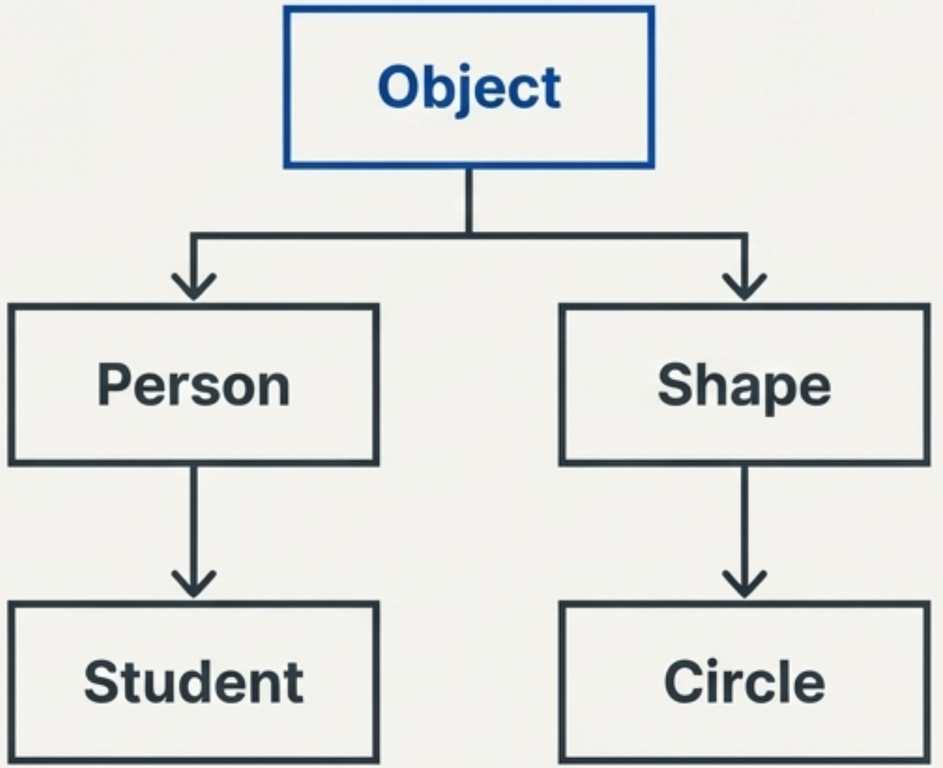


The Hidden Ancestor: The `Object` Class

The Root of Everything

In Java, every class you create is a descendant of the `Object` class, whether you write `extends Object` or not. It is the ultimate superclass at the top of every inheritance tree.

This means every single object in Java inherits a set of fundamental methods from **Object**, such as **toString()**, **equals()**, **hashCode()**, and **getClass()**.



All classes in Java ultimately inherit from the `Object` class.

Customizing the Universal Toolkit

Overriding `Object` Methods

The default methods from `Object` are often not useful. We should override them:

- `toString()`: Provide a readable, string representation of your object. Essential for debugging and logging.
- `equals(Object obj)`: Define what it means for two objects of your class to be logically equal (e.g., two `StudentCard` objects with the same ID).
- `hashCode()`: Must be overridden whenever `equals()` is. It provides a numeric representation used in collections like HashMaps.

W11E09.java

```java
class StudentCardE09 { // Implicitly extends Object
  private final String id;
  private final String name;
  // ... constructor ...

  @Override
  public String toString() {
    return "StudentCard[id=" + id + ", name=" + name + "]";
  }

  @Override
  public boolean equals(Object obj) {
    // ... logic to compare by 'id' ...
  }
}

StudentCardE09 card1 = new StudentCardE09("S123", "Hana");
System.out.println(card1); // Prints the custom toString() output!
```

Verifying Type in a Polymorphic World

`instanceof` vs. `getClass()`

Sometimes you need to check an object's type before performing a specific action. Java provides two ways to do this:

- **The `instanceof` Operator:**
  Checks if an object is an instance of a class OR any of its subclasses.
  Returns `true` for (`new Student() instanceof Person`).
  Use this for general type checking: "Is this object a kind of Shape?"
- **The `getClass()` Method:**
  Inherited from `Object`; cannot be overridden.
  Returns the object's exact runtime class.
  `obj1.getClass() == obj2.getClass()` checks if they are instances of the *exact same* class.
  Use this for strict equality checking: "Is this object specifically a Circle, not just a Shape?"

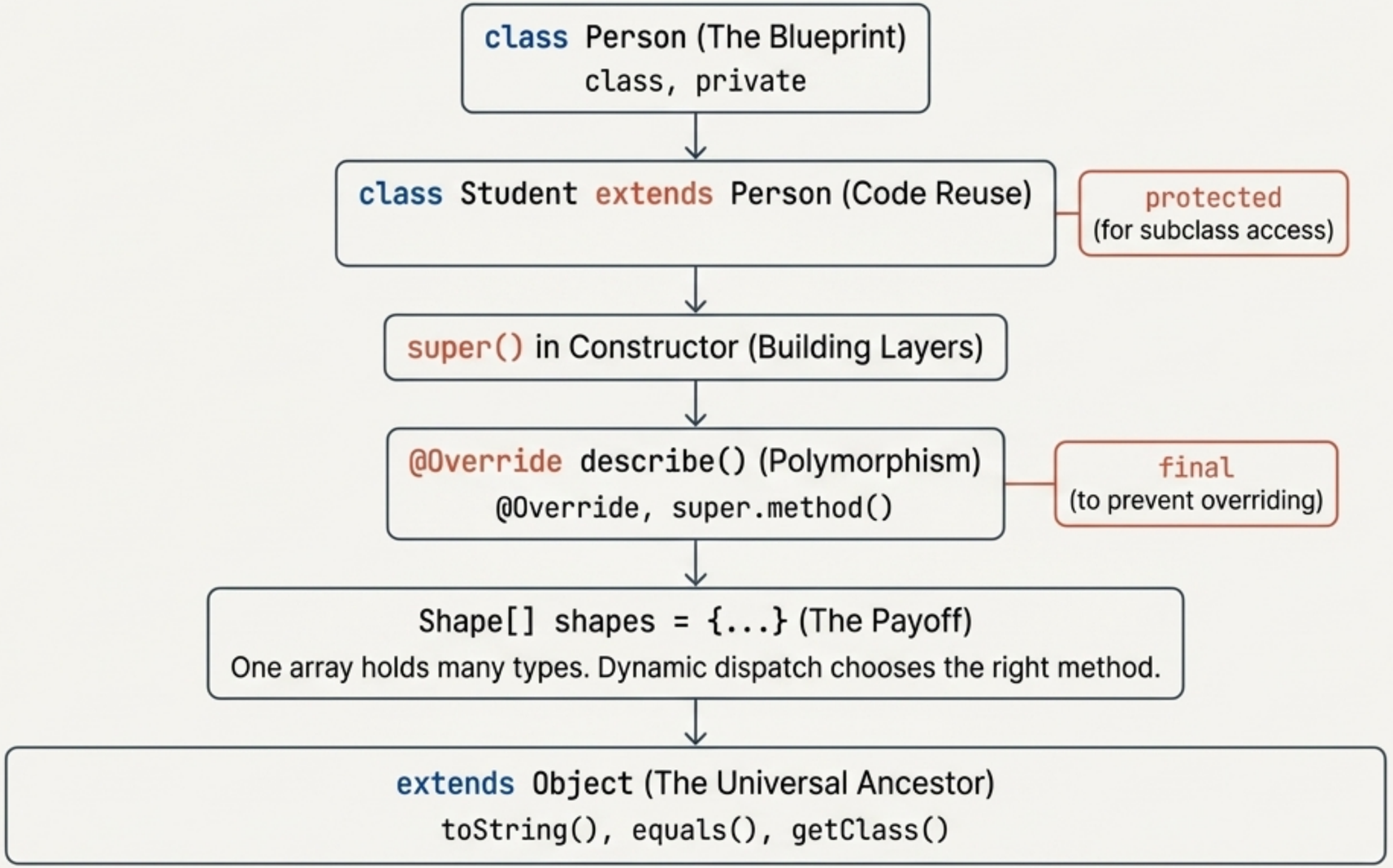
Code for `instanceof`

```
Object obj = new StudentE04("Farid", "Math");

if (obj instanceof StudentE04) { // true
   System.out.println("It's a Student.");
}
if (obj instanceof PersonE04) { // also true!
   System.out.println("It's also a Person.");
}
```

Code for `getClass()`

```
Object p1 = new PersonE04("Elaine");
Object s1 = new StudentE04("Farid", "Math");

if (p1.getClass() == s1.getClass()) { // false
    System.out.println("They are the same class.");
}
```

The Inheritance Toolkit: A Summary



Inheritance is a core pillar of OOP that enables code reuse, specialization, and the creation of flexible, maintainable software.